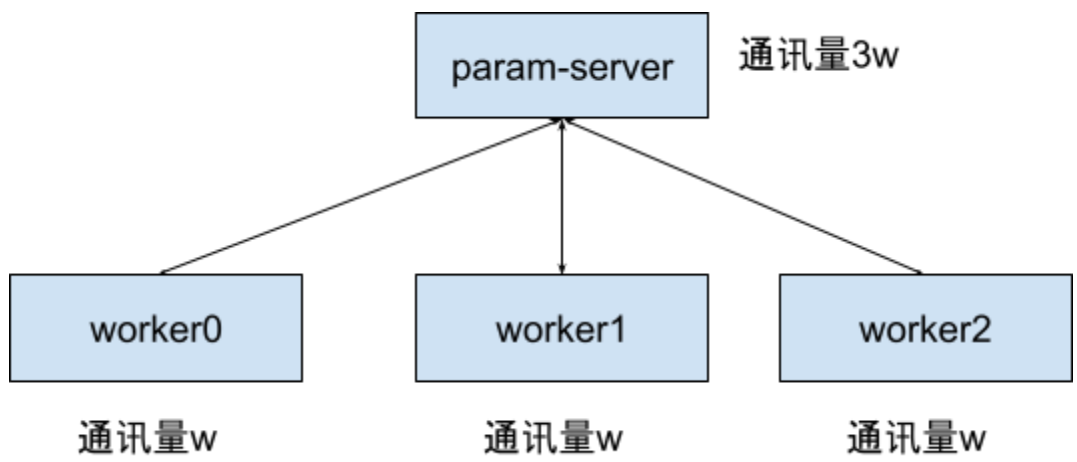
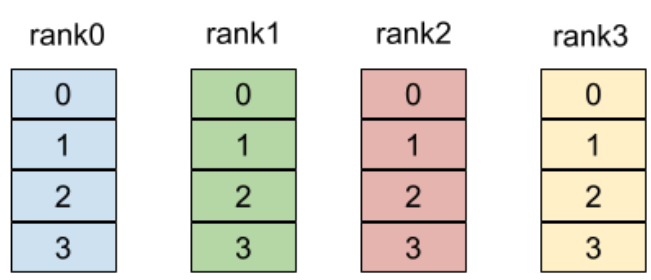
版本	修改人	时间
v.01		20241115

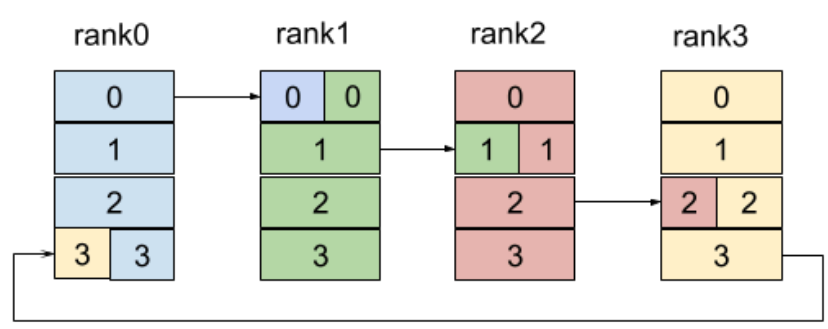
NCCL背景

PS-Worker/AllReduce变化

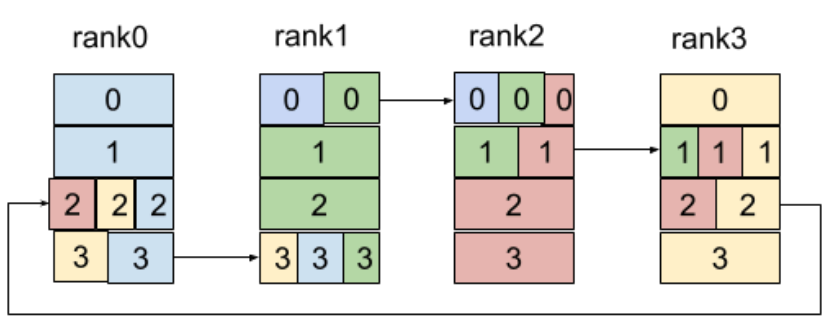


pserver模式下，每个worker将数据发送到param-server算平均梯度并同步，因此param-server通讯量是3w，随着训练规模的扩大，param-server会成为瓶颈

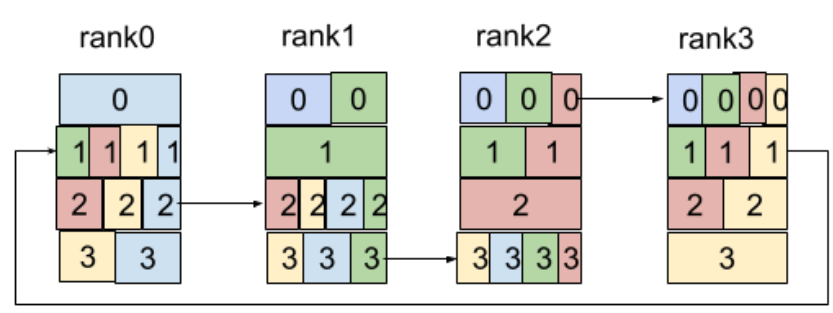




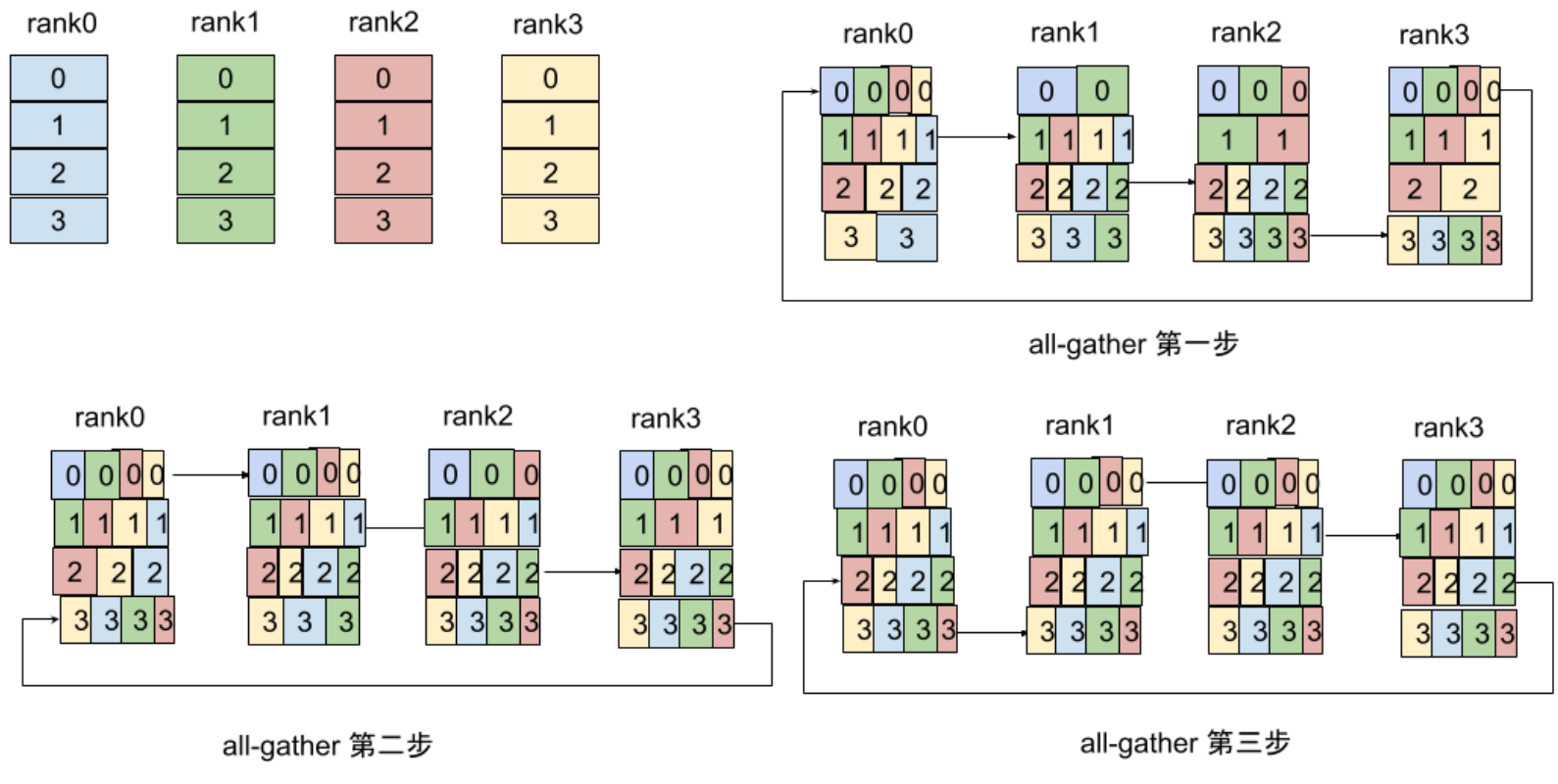
scatter 第一步



scatter 第二步



scatter 第三步

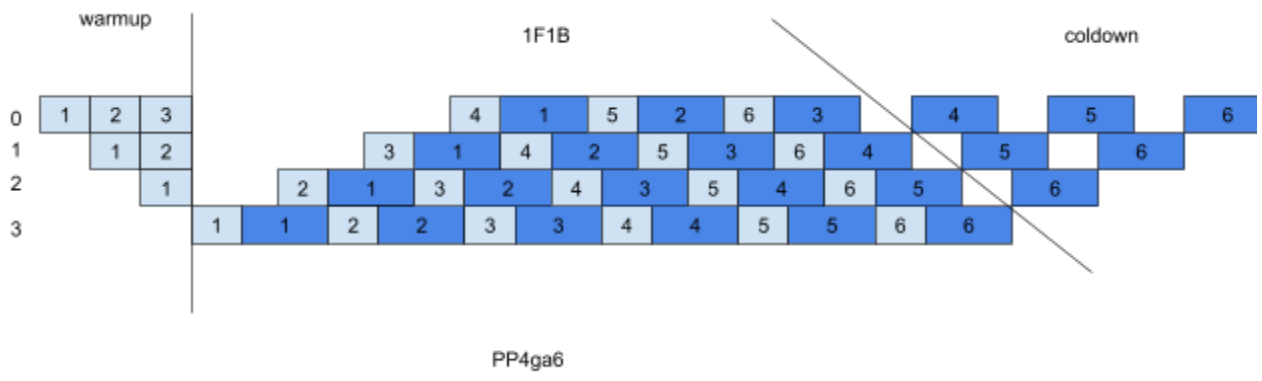


通过allreduce这种形式，每个节点只需向其他节点发送(n-1)次数据，每次发送w/n数据，重要的是每个节点发送的数据量相同。
有了pserver到ring的allreduce，可以看到的变化是，从所有节点都向一个节点通讯，变成所有节点向其他节点发送等量的数据，结束了性能瓶颈，但是随着GPU数量的增多，环会变得很长，延迟会变高，从而影响带宽，然后有人提出了double-tree，从数据结构上来说是把list变成tree，将延迟从N变成lgN，将影响延迟的因素从环的长度变成树的高度

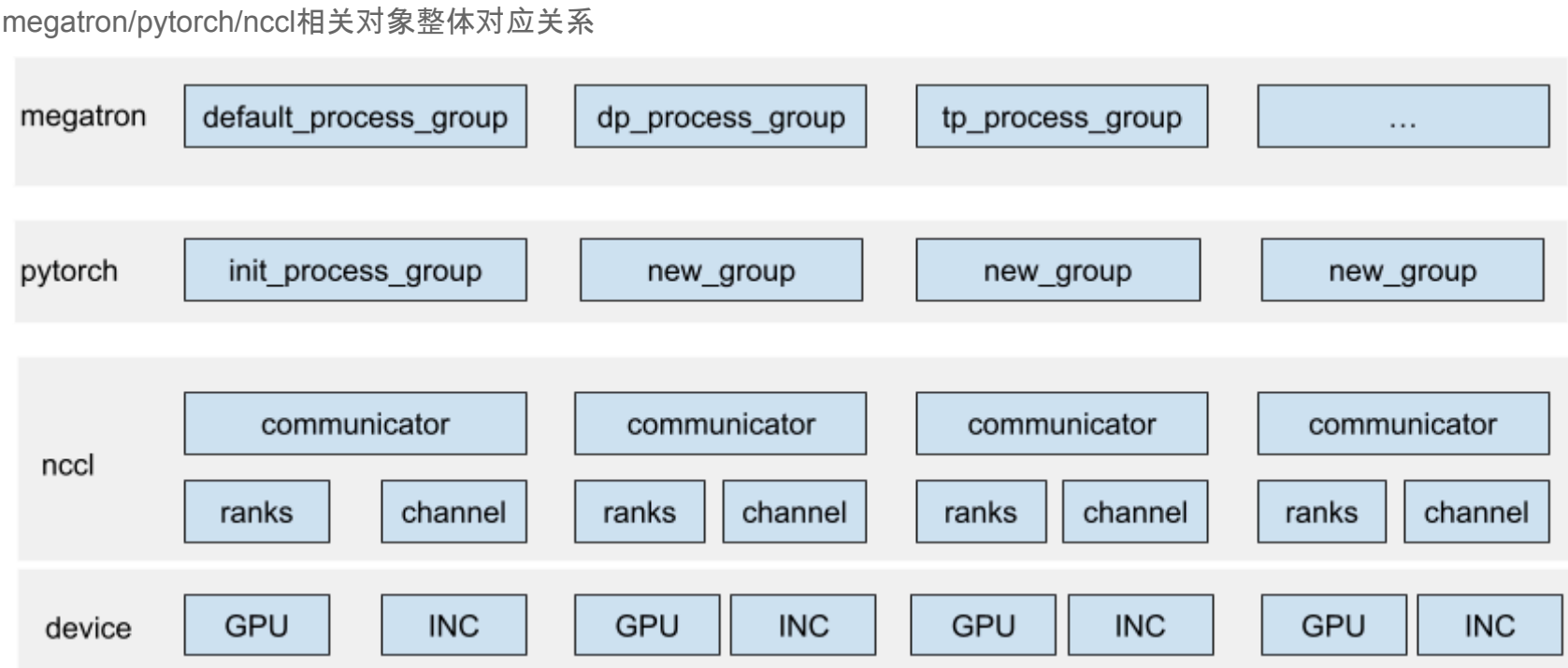
训练切分方式

一切为了加速训练

batch size大小会影响模型最终的准确性和训练过程的性能，而数据的batch size大小受到GPU内存限制，于是有人提出梯度累计，就是一次没法训练太多的数据，将多步的训练结果合成一步，然后后来随着模型的增大，GA也解决不了，出现了将完整的模型放到不同的GPU上，每个GPU训练的数据不同，这个就是数据并行；后来模型结构越来越大，导致一个GPU放不下完整的模型，就出现了TP/PP，一个是切分tensor，一个是切分layer，这里就出现了另外的问题，将tensor和layer切分之后，很多操作需要完整的模型，所以这里涉及到了通讯问题，具体怎么处理，torch的处理方式是给每个切分方式设置一个process-group，nccl的处理方式是将不同的切分方式单独做成一个communicator，在这个comm中该种切分方式不同rank之间通讯
PP切分方式



megatron/pytorch/nccl相关对象整体对应关系



Rank概念以及rank如何确定

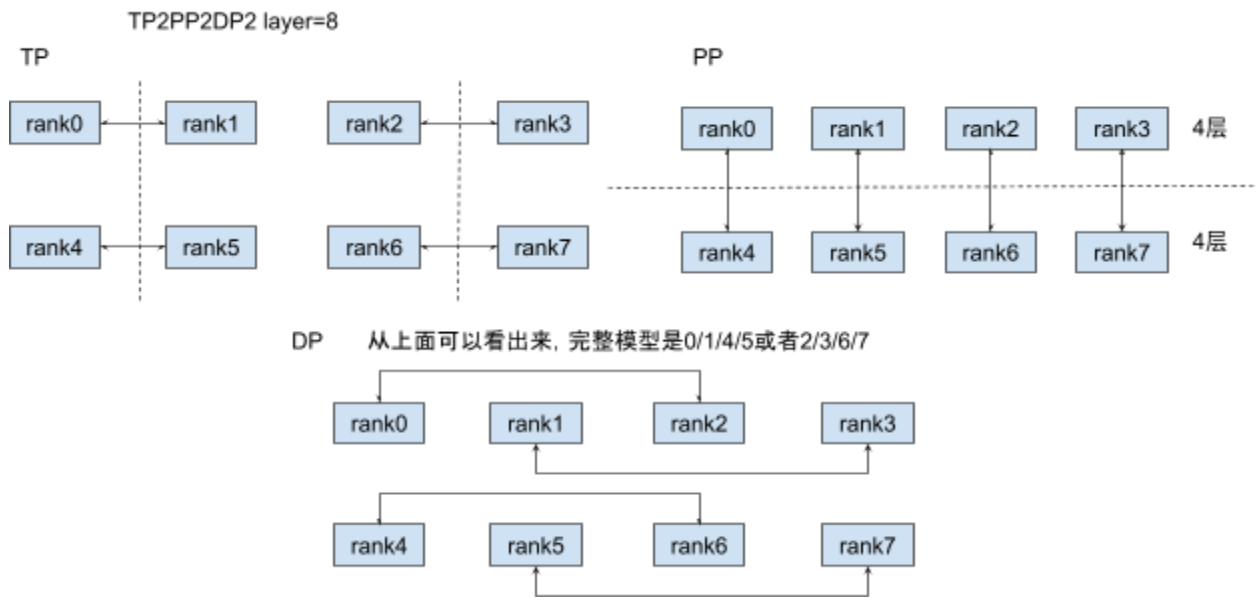
Rank的定义

rank一开始是在MPI中引入的，表示的是一个进程，这里表示给一组进程中的每个进程编号，现在的训练中由于一个训练进程运行在一张GPU上，所以一个rank也代表着一张GPU卡

Rank如何确定

详细信息可以参考 nccl_rank选择

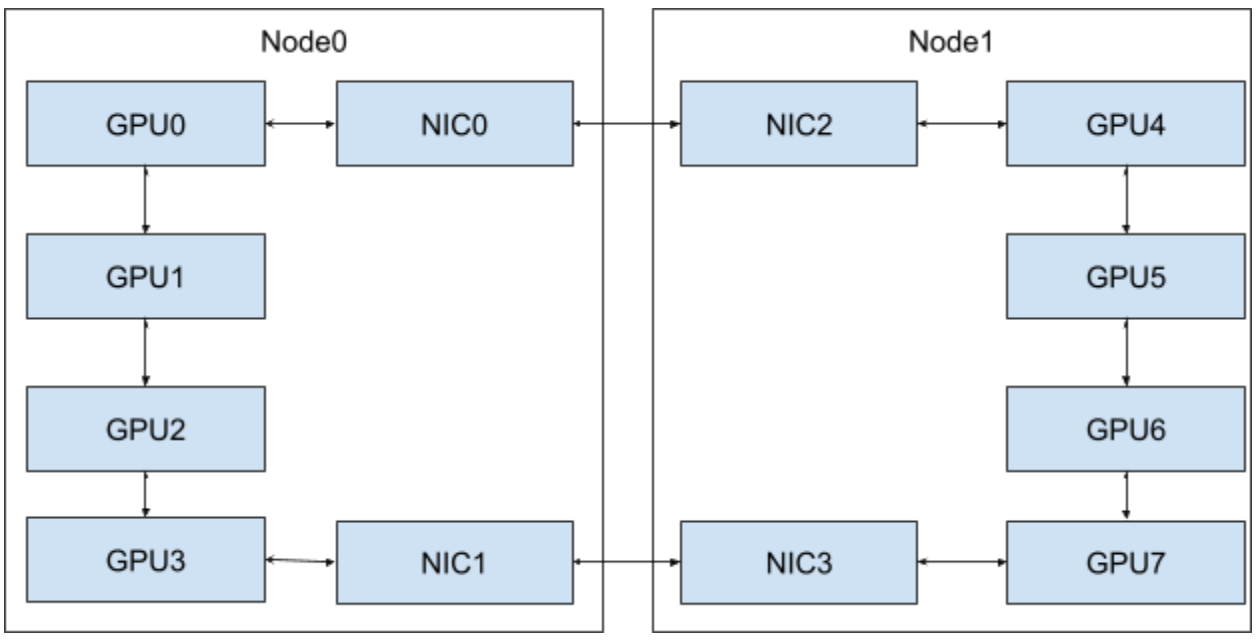
切分以及rank分组例子



其中分到一组的rank会进行通讯，真实通讯的时候是按照rank编号来的，跟GPU号等其他的不直接相关
拿TP来说，TP2表示将tensor分成两份，也就是说每两个GPU可以组成完整的tensor，对应8个rank的具体分组上，[0,1], [2,3],[4,5],[6,7]四组

NCCL目标

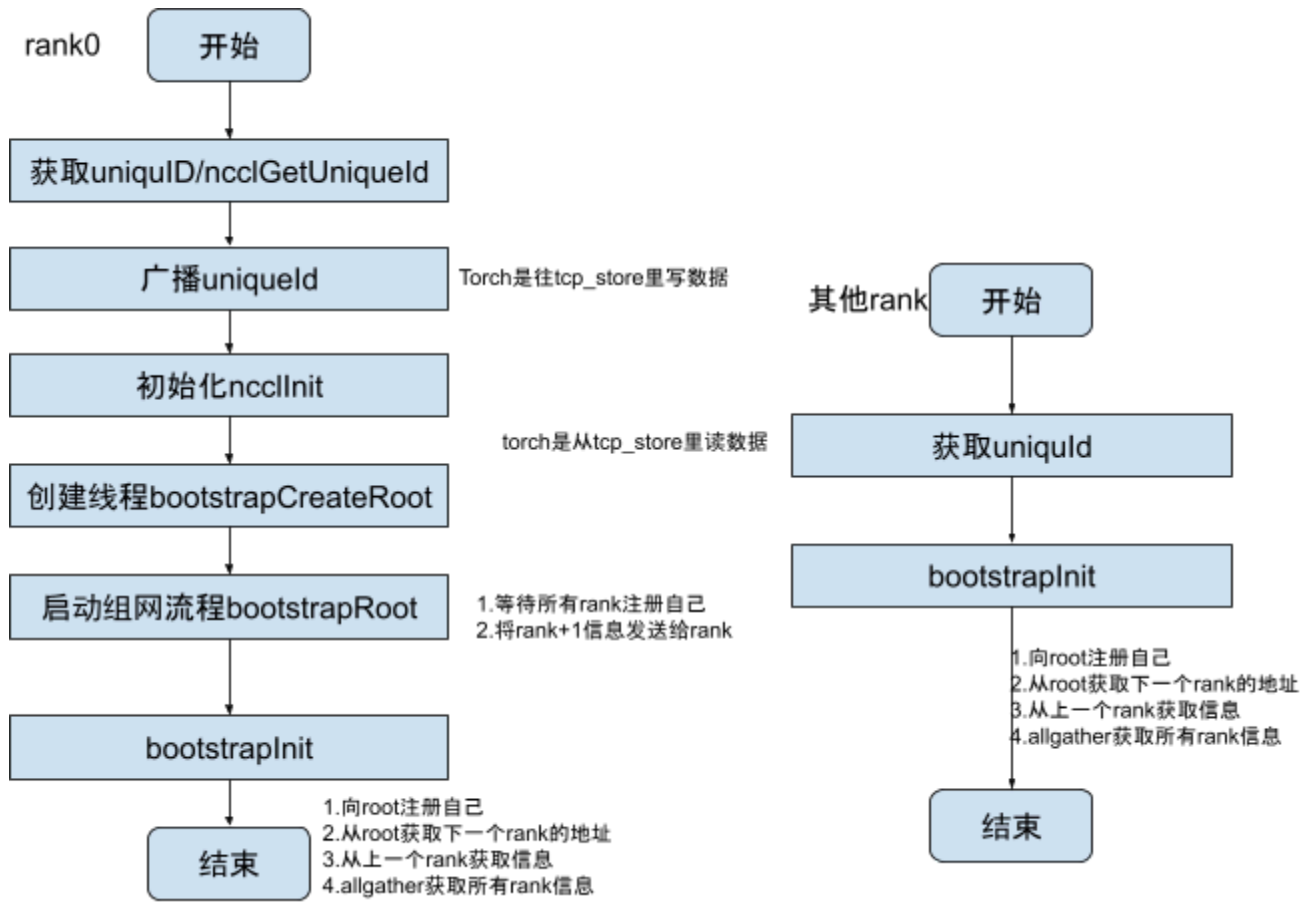
NCCL的是nvidia collective communications Library的缩写，是一个通讯库，**目标是能完全利用设备带宽，提高设备之间的吞吐**
如下图所示，两个节点Node0/Node1，每个节点四张GPU两张网卡，当从Node0和Node1进行通讯的时候，如果把两张网卡都用起来并发发送数据那么目标就达到了，为了做到这一点，nccl将完全独立的物理设备建立数据通讯链路，具体做法是在通讯之前会先把数据发送的通道找出来，然后建立通讯，当进行数据通讯的时候，通过这些链路收发数据



NCCL解析

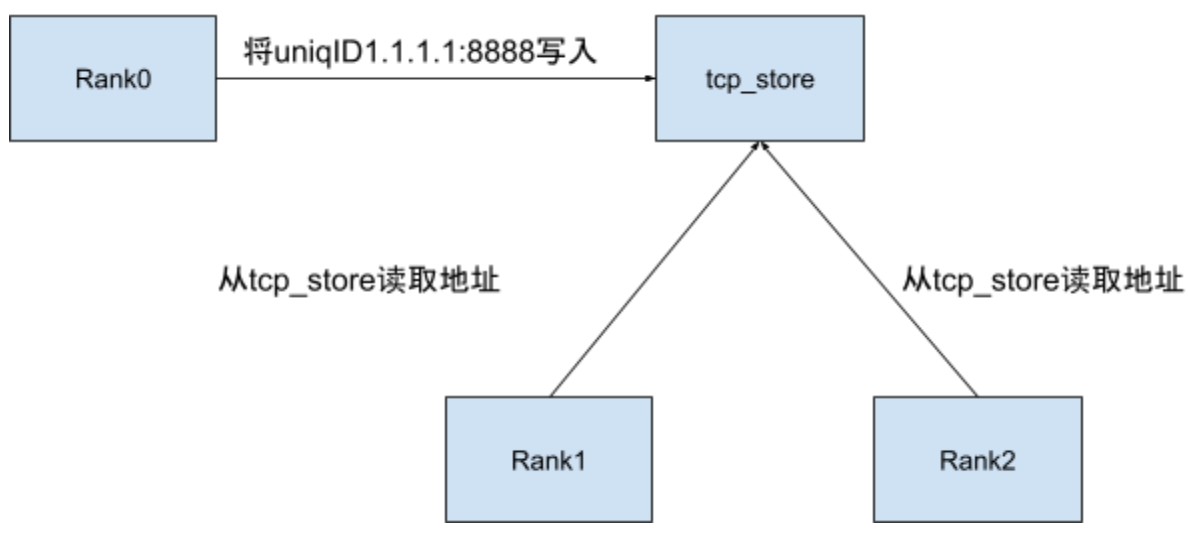
Bootstrap组网信息构建

这个是类似于服务注册发现的过程



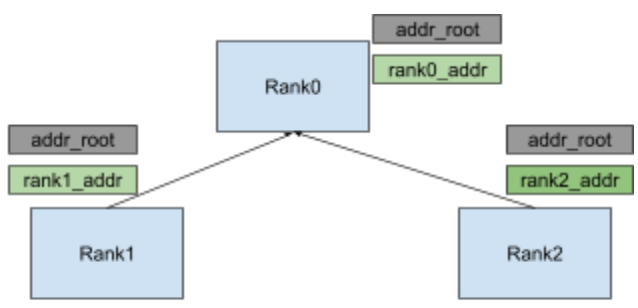
其中这里的uniQueId其实就是master的连接地址

第一步确定rank0地址

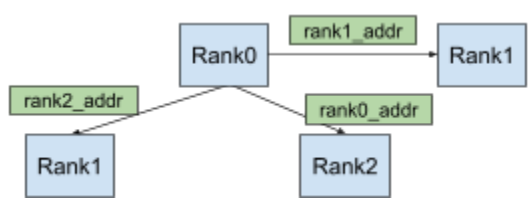


第二步同步各个Rank连接信息

1.所有rank将自己信息发给rank0

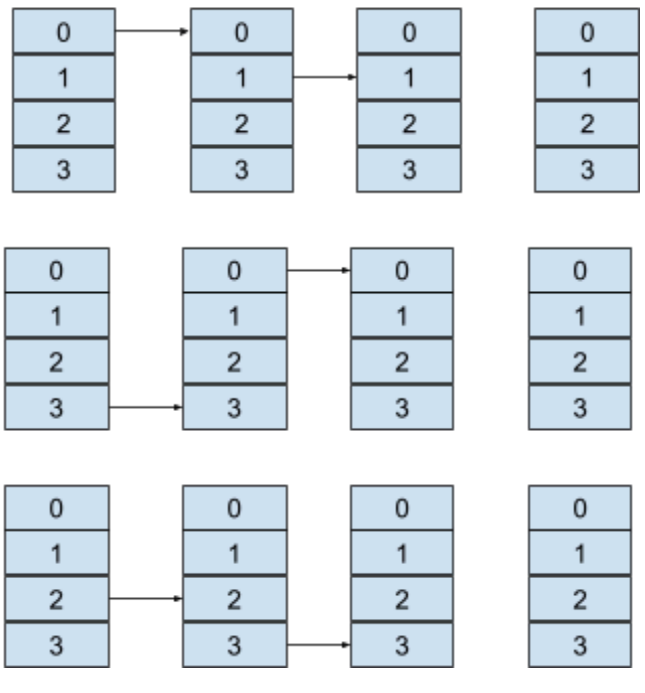


2. rank0将rankN的信息发给rankN-1,
所有的rank获取到下一个rank连接信息之后会建立连接



第i步接受前一个节点rank-i-1
发送上一步获取的rank-i的数据发送给下一个rank

3. Allgather 获取所有rank的连接信息



bootstrapRingAllGather
这个是Rank1的发送数据的过程
// i nrank rank rslice sslice
// 0 4 1 0 1
// 1 4 1 3 0
// 2 4 1 2 3
// 3 4 1 1 2

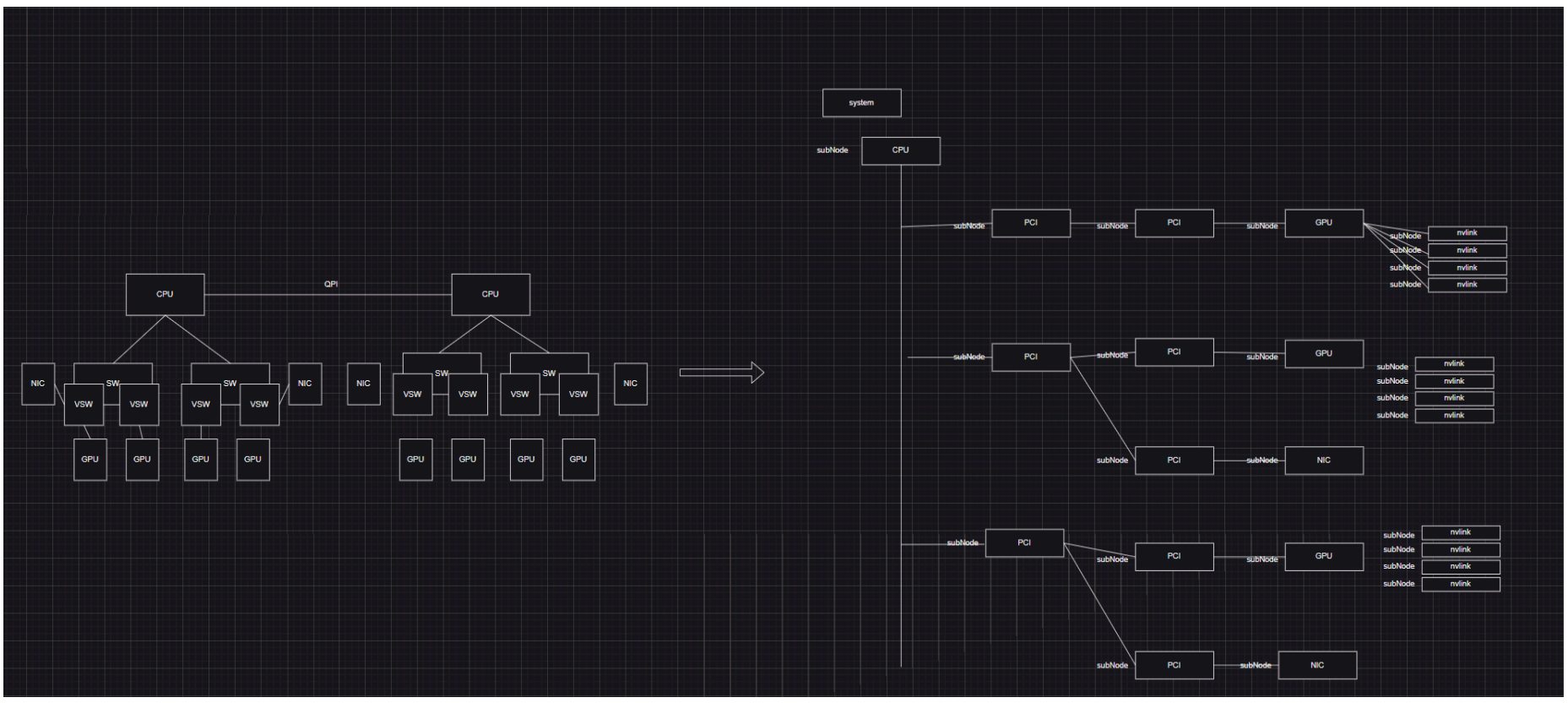
每个Rank经过n-1次收发之后，获得各自的数据

topo构建和grap搜索

获取Topo

获取xml

这一步是将物理拓扑转换成一个树状结构



左边的是物理结构，右边是将物理结构转换成树状结构，一个node下面有subs变量表示子节点，parent表示父节点，由此串联成树状结构

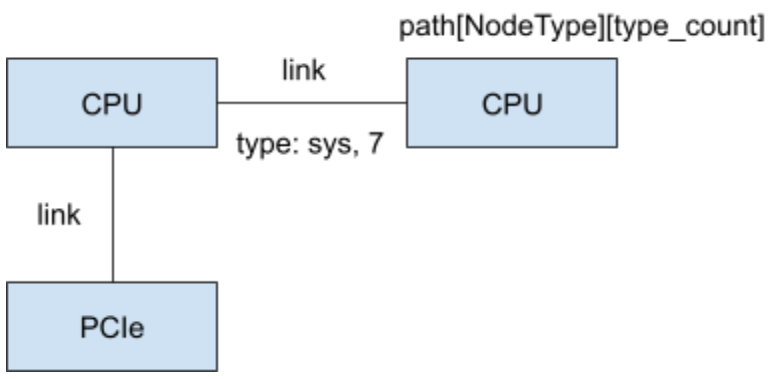
```
struct ncclXmlNode {
 char name[MAX_STR_LEN+1];
 struct {
   char key[MAX_STR_LEN+1];
   char value[MAX_STR_LEN+1];
 } attrs[MAX_ATTR_COUNT+1]; // Need an extra one to consume extra params
 int nAttrs;
 int type;
 struct ncclXmlNode* parent;
 struct ncclXmlNode* subs[MAX_SUBS];
 int nSubs;
};

struct ncclXml {
 int maxIndex, maxNodes;
 struct ncclXmlNode nodes[1];
};
```

其中每个设备都有对应的属性，放到key和value中，type表示设备类型

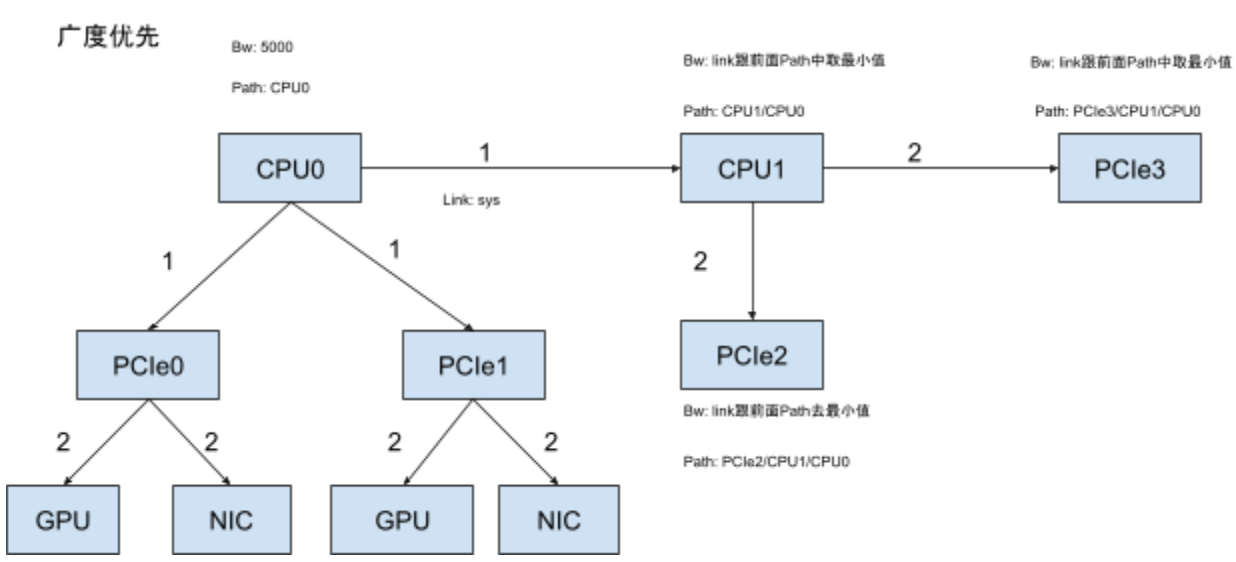
构建topo

这一步是先构建一个无向图，之后通过广度优先遍历获取节点间带宽最大并且距离最短的路径，这里构建出来的路径后面搜索channel的时候会使用
先来看下Node对象，topo就是不同设备之间是如何连接的，他们之间带宽多大；一个节点到另一个节点的最近距离；后面找channel的时候，每找到一条channel会把带宽剪掉，网卡没有这么做，因为网卡剪掉没法搜索了。通过上面获取到的xml逻辑结构树，其中subs表示子节点，这里把子节点可以理解成连接到的设备，放到links字段中，path表示当前设备到其他设备的最近距离，最关键的path的搜索过程，得到path之后，会在后面channel搜索的时候用到



这里说下link类型

```
#define LINK_LOC 0
#define LINK_NVL 1
// Skipping 2 for PATH_NVB
#define LINK_PCI 3
// Skipping 4 for PATH_PXB
// Skipping 5 for PATH_PXN
// Skipping 6 for PATH_PHB
#define LINK_SYS 7
#define LINK_NET 8
```



以CPU为例，从topo树中依次遍历相邻的设备，并将path信息写入对方的path中，当直接相邻的设备遍历完之后，会把这些节点放入待遍历的节点，然后依次遍历这些节点直接相连的设备，依次类推，这个是典型的广度优先遍历
Path类型

```
// Local (myself)
#define PATH_LOC 0
// Connection traversing NVLink
#define PATH_NVL 1
// Connection through NVLink using an intermediate GPU
#define PATH_NVB 2
// Connection traversing at most a single PCIe bridge
#define PATH_PIX 3
// Connection traversing multiple PCIe bridges (without traversing the PCIe Host Bridge)
#define PATH_PXB 4
// Connection between a GPU and a NIC using an intermediate GPU. Used to enable rail-local, aggregated network send/recv
operations.
#define PATH_PXN 5
// Connection traversing PCIe as well as a PCIe Host Bridge (typically the CPU)
#define PATH_PHB 6
// Connection traversing PCIe as well as the SMP interconnect between NUMA nodes (e.g., QPI/UPI)
#define PATH_SYS 7
// Connection through the network
#define PATH_NET 8
// Disconnected
#define PATH_DIS 9
```

Graph搜索

网卡选取

这里首先来看网卡选取，是因为机间的通讯，第一步是遍历GPU，获取最近的网卡放到数组中，之后遍历网卡数组，搜索channel，相关的方法是ncclTopoSelectNets

逻辑：选择离GPU最近的网卡，当出现距离相同的网卡时，按照一定规则筛选，类似做了负载均衡，这里说负载均衡是因为跟channel有关，不同的channel选择不同的网卡

```
for gpu 0-8:
    for channel_num 0-max:
        ncclTopoGetLocalNet:
```

ncclTopoGetLocalNet

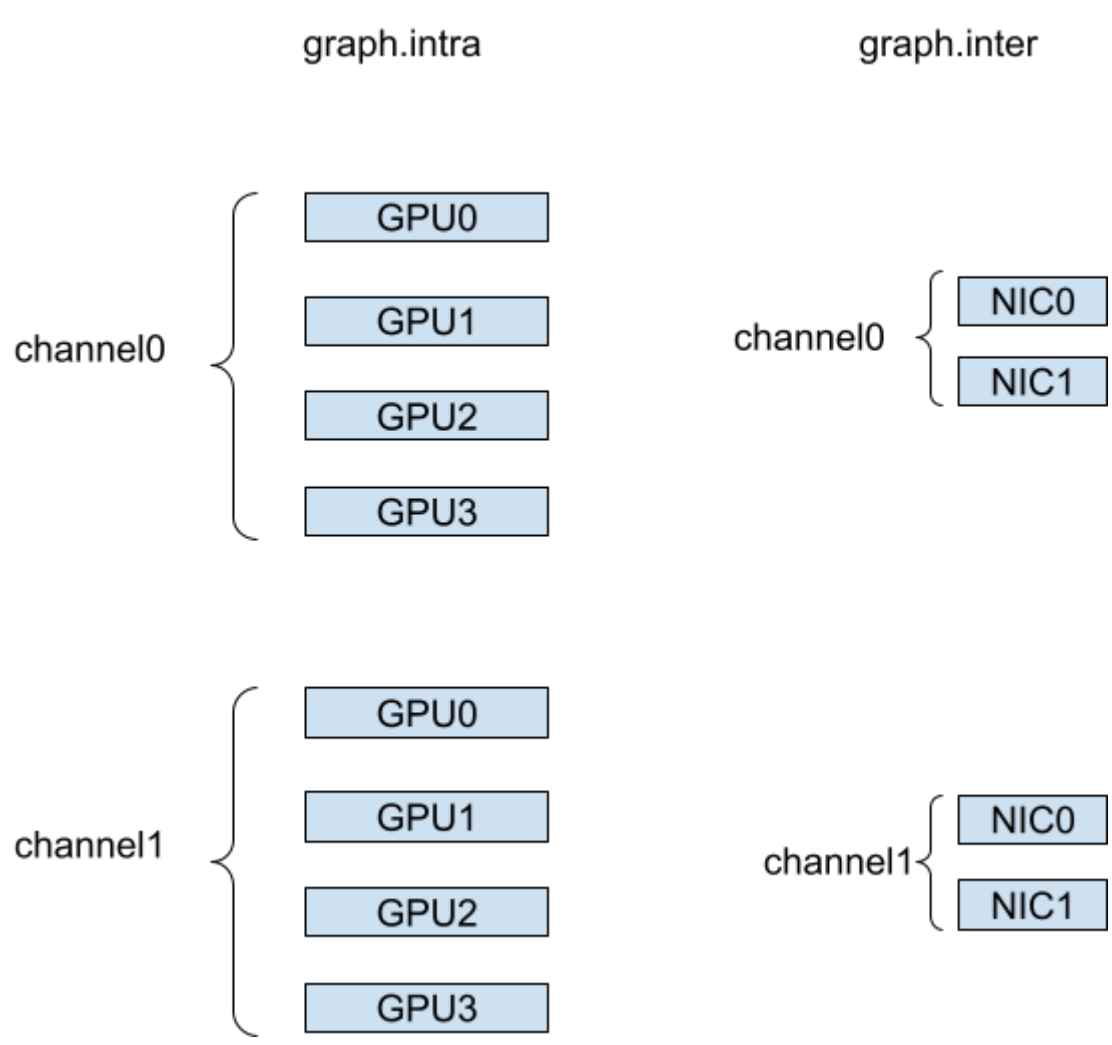
1. 把rank转换成index，这里的index是GPU在system中顺序的编号，而这里的rank对应的是Rank
2. ncclTopoGetLocal，获取离GPU最近的网卡
    1. 获取网卡数量
    2. 获取当前GPU到其他网卡的Path信息，system->nodes[type].nodes[index].paths[resultType]
    3. for 网卡0-4：
        1. 从path中找到四个网卡中bw最大并且跳数最少的，如果带宽跟跳数相同，就把多个网卡都放进去，否则只保留一个
3. ncclTopoGetLocal，获取离上一步中获取的第一张网卡最近的所有GPU
4. 这里将当前GPU的设备编号作为网卡编号net
5. 负载均衡
    1. if (isPow2(localNetCount)) net = mirrorBits(net, localNetCount); net += channelId%(DIVUP(localNetCount,localGpuCount))；这里获取到的网卡数除以获取到的GPU数并向上取整获得数据A，然后拿channelId%A，获取网卡
6. 拿上一步获取到的网卡号%获取到的网卡数量，作为最终选择的网卡

总体逻辑

入口是在ncclTopoCompute，它的目标是寻找节点内的最大并发路径数，并且选择最大的带宽，主体逻辑是在ncclTopoSearchRec，先看下外层的逻辑，然后里面的逻辑是一个有条件的暴力搜索，像贪心算法
如果第一遍ncclTopoSearchRec没有搜到合适的channel，就降低要求，再去搜索，直到搜索到一个解决方案，如果找到一个解决方案，尝试寻找带宽更大的解决方案。

Ring搜索

为了好理解，我们先看下数据结构，这里主要有两个，inter和intra，inter表示机间通讯，里面放的是网卡，intra表示机内通讯，里面放的是GPU；然后整体通讯就是把inter和intra连起来构成



搜索过程

机内

对于ring来说，搜索的目标是每一个channel由进出节点的两张网卡+选择的GPU构成，

1. for i in nets 遍历所有网卡

a. 将当前网卡放入graph->inter[graph->nChannels*2], 就是作为当前channel的起始网卡
b. 对于第一个channel, 找到带宽最大的GPU, 从这个GPU开始搜索
c. 对于其他channel, 通过重放获取GPU, 然后指定选择下一个GPU策略是重放, 也进行路径搜索

之后是ncclopoSearchTryGpu到ncclTopoSearchRecGpu, 直接看后者, 前者只是后者添加了增减带宽处理, 核心在后面。
ncclTopoSearchRecGpu, 这个方法是遍历所有的GPU, 选择合适的GPU放入graph->intra, 之后选择合适的网卡放入graph->inter,这里网卡是结尾, 开头是在上面一开始遍历网卡的地方
方法解析:

```
If step == gpu_count: // 当走到步数等于gpu数量时, 这个时候需要结算了
   graph->nChannels++;
   ncclTopoCompareGraphs(graph, tmp);
   If tmp是一个更好的channel:
      将tmp复制到graph;
   If graph->nChannels < graph->maxChannels:
      ncclTopoSearchRec; // 继续递归搜索channel
   graph->nChannels–; // 暴力搜索
将当前gpu放到graph->intra; // 这里是核心的一步
If step == backToNet: // 当步数等于gpu数量-1时, 说明GPU已经选完了需要回到网卡, 这里要选择网卡
```

1. 以当前GPU作为起点, 找到所有的网卡, 调用的是前面的是ncclTopoSelectNets,
2. 遍历所有的网卡,
   a. 如果当前GPU到某个网卡的带宽满足要求, 就将该网卡放入graph->inter[nChannels*2+1], 开始搜索下一个channel
      i. 需要注意的是, 带宽扣减在ncclTopoFollowPath/followPath方法里
   b. 如果没有找到当前GPU满足带宽要求的网卡, 本次搜索结束, 回退

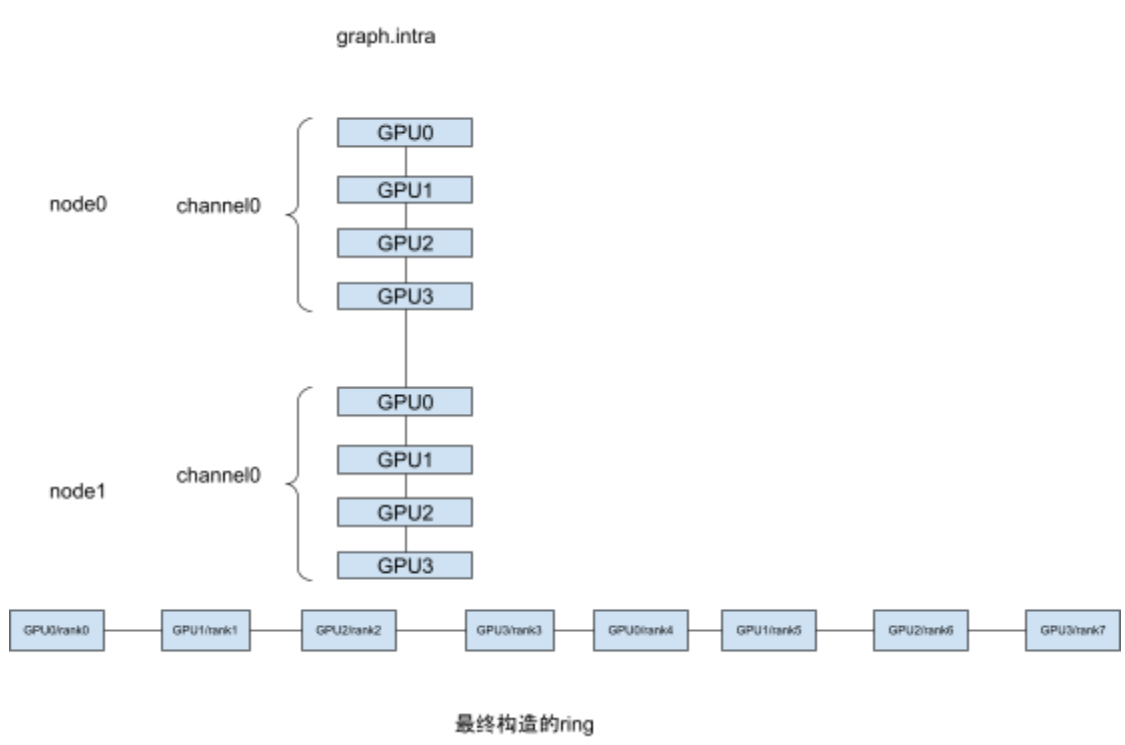
```
If step < system->nodes[GPU].count-1: // 这里是继续查找下一个GPU
   If 按照PCI顺序:
      下一个GPU就是当前的step+1;
   If 选择重放:
      从graph->intra中获取上一次channel的下一步的GPU编号
   else 常规情况:
      ncclTopoSearchNextGpuSort, 该方法是对GPU排序, 作为下一步的GPU来选择
```

1. 根据当前GPU的paths, 获取其他GPU到当前GPU的距离+带宽等信息, 如果下一步是网卡, 还要加上网卡的带宽
2. 根据机间网卡带宽, 机间Pci带宽, 机间跳数, 机内带宽, 机内跳数, GPU编号等条件对GPU排序
3. 按照排好的顺序将GPU放入next数组
4. 如果系统中存在nvswitch, 优先选择跟当前GPU序号相连的两张卡

```
   For i in gpu_count: 对后续的每一个GPU进行搜索尝试
         ncclTopoSearchTryGpu, 这个方法获取对应的路径设备, 之后会调用ncclTopoSearchRecGpu, 暴力递归
```

机间

从实现上来说, 一共三步, 将graph->intra放入ncclTopoRank; allgather获取所有节点的graph信息; 将所有设备连接



最终构造的ring

Ring的机间起始比较简单, 就是把每个节点的channel相连;
至于为什么非要搞prev/next, 这个起始跟通讯有关系, 因为ring通讯的时候就是就上一个rank和下一个rank通讯, 是为了后面通讯收发数据方便
至于数据发送这一块, 我们这一版先不写, 后面再补充;看上去graph->inter这里没有使用, 在构建send/recv连接的时候, 会选择使用的网卡, 那个时候会用到, 具体在ncclTopoGetNetDev

Tree搜索

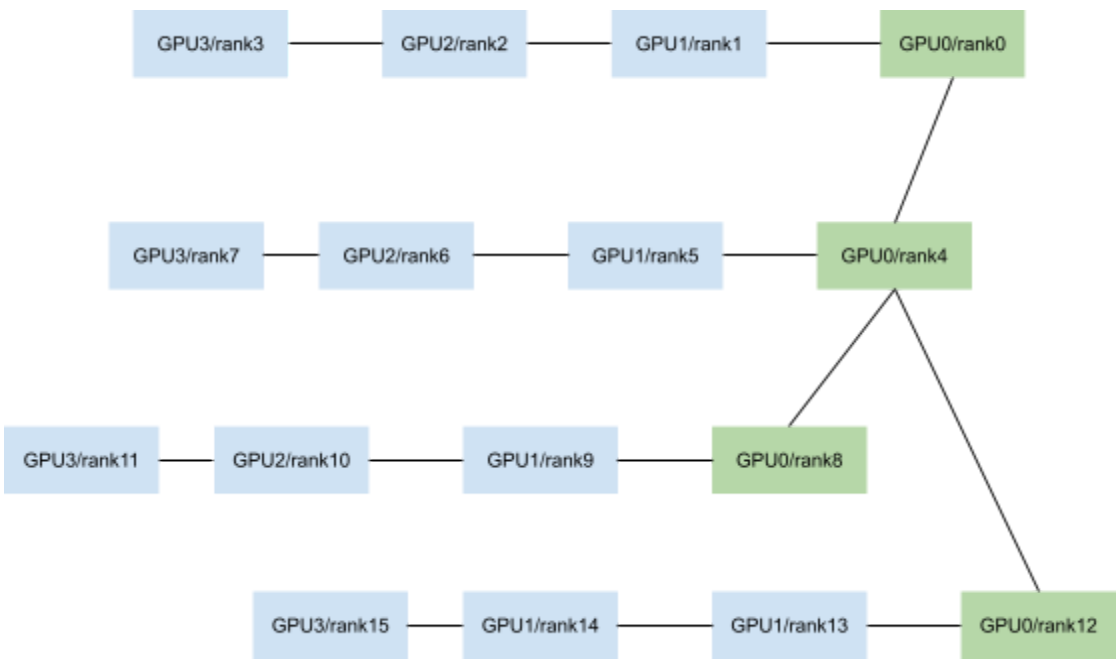
tree前面提到过为了降低GPU规模带来的延迟增加问题, 改用double-tree, 目前的实现是相当于每个机器相当于tree中的一个节点(对于简单的tree可以这么理解), 节点内也是一条链, 至于为什么这么实现, 猜想是节点内带宽较大, 延迟不会超过节点间, 目前这种实现比较简单, 看官方的测试效果也不错

机内

tree分为三种NCCL_TOPO_PATTERN_TREE/NCCL_TOPO_PATTERN_SPLIT_TREE/NCCL_TOPO_PATTERN_BALANCED_TREE这里先讲最简单的NCCL_TOPO_PATTERN_TREE，他跟ring的机内搜索差不多，其实三种都差不多，差别都是在选择网卡上，跟ring处理差别是在转换成topoRank的时候，才会展现出tree的特性

- 的NCCL_TOPO_PATTERN_TREE要求进出网卡要是同一个

机间



通讯时如何选择Channel

通过带宽计算选择带宽最好的channel，这里有两个纬度，一个是通讯原语，一个是算法，对于不同的通讯原语会根据带宽选择不同的算法，举个例子，tree只是实现了allreduce，也就是说就算tree性能好，很多操作无法使用

目前通讯原语和算法支持统计

通讯原语/算法	ring	tree	
send	✓	✘	
recv	✓	✘	
reduceScatter	✓	✘	
reduce	✓	✘	
broadcast	✓	✘	
allreduce	✓	✓	
allgather	✓	✘	

带宽计算：

```
ncclResult_t ncclTopoGetAlgoTime(struct ncclInfo* info, int algorithm, int protocol, int numPipeOps, float* time) {
 float bw = info->comm->bandwidths[info->coll][algorithm][protocol];
 float lat = info->comm->latencies[info->coll][algorithm][protocol];
 if (bw == 0) {
   *time = -1.0; return ncclSuccess;
 }
 int logSize = log2i(info->nBytes>>6);
 if (algorithm == NCCL_ALGO_TREE && logSize < 23) bw *= treeCorrectionFactor[protocol][logSize];
 if (info->nChannels != 0) bw = bw / info->comm->nChannels * info->nChannels;
 if (algorithm == NCCL_ALGO_RING && protocol == NCCL_PROTO_SIMPLE && info->comm->nNodes > 1
     && info->coll == ncclFuncAllReduce && info->nBytes/(info->comm->nChannels*info->comm->nRanks) >= 64) {
   lat *= info->comm->minCompCap < 80 ? 1.9 : 1.4; // Plateau effect of ring
 }
 // Tree pipelining saves latency in aggregation cases
 int latCount = algorithm == NCCL_ALGO_RING ? numPipeOps : DIVUP(numPipeOps, NCCL_MAX_WORK_ELEMENTS);
 *time = lat * latCount + (info->nBytes) / (1000 * bw);
 return ncclSuccess;
}
```